An Empirical Analysis of Multi-Agent Systems for Football Data Analytics and Performance Prediction

Ethan Bensadoun[1], Lorenzo Ravera[1], Till Tornieporth[1], Marcos Barreto[1], and Huma Lodhi[2]

[1] London School of Economics and Political Science, London, UK
[2] Sky, UK

Abstract. Recent advances in the reasoning capabilities of Large Language Models (LLMs), together with a rapid increase in available football data, create a new opportunity for agentic systems to provide data-driven insights for complex football queries such as match analysis, strategy, and outcome or league-rank prediction. Football analytics is dynamic and multi-player, which raises a central question: whether and when to use a Multi-Agent System (MAS) rather than a single LLM. Prior research suggests that multi-agent decomposition improves accuracy on complex, multi-step tasks. We test this in the football setting by comparing a MAS against a single LLM with identical tool access and a single LLM with only raw data access, isolating the effect of orchestration from the effect of tool access. We evaluate accuracy, token usage, and inference time on a ground-truth set of 50 Premier League queries. We find that the autonomous MAS variant with Claude Opus 4.8 performs best on out-of-coverage queries and use it for subsequent experiments. The MAS outperforms a single LLM with the same tool access (89% vs. 83%), and both outperform a single LLM with only raw data access (35%). Structured database access is the dominant driver of accuracy, well above the gain from orchestration. We also show that adding a persistent memory layer yields clear cost savings on recurring queries but no accuracy benefit on independent ones, and that naive similarity-based caching is unsafe for temporal data, which motivates structured, parameter-aware reuse.

Keywords: Multi-Agent Systems · Large Language Models · Retrieval-Augmented Generation · Football Analytics · Match Outcome Prediction

1 Introduction

Large language models have become the interface layer for data-driven analysis, letting users with little statistical background turn natural language into database queries that extract evidence and synthesise explanations [16]. Sports analytics is a natural fit: the domain is rich in structured data, yet a fan, journalist, and analyst all query the same underlying data regardless of their question [9]. A conversational agent that can answer, which team had the best goal

difference last season, or forecast an upcoming fixture lowers the barrier between raw football data and the people who reason about it.

A growing body of work, including AutoGen [13] and MetaGPT [5], argues that multi-agent systems (MAS) outperform single monolithic LLMs on complex, multi-step tasks. The benefit does not stem from scale but from orchestration, tool delegation, and error isolation, codified agentic design patterns [4] we apply to football analytics.

Crucially, this MAS-versus-single-LLM comparison has not been tested on football data. Football analytics research is dominated by machine-learning models for specific predictive tasks: expected goals, match-outcome forecasting, player valuation, injury risk, each a bespoke model evaluated in isolation [3]. These do not address how best to organise an LLM-driven analytics system over football data, nor whether multi-agent decomposition transfers to this domain.

Realising these benefits requires several established practices, which we adopt and evaluate. Retrieval-augmented generation and a domain knowledge base ground outputs in authoritative data rather than the model's parametric knowledge. The control-flow architecture, where branches are wired and how failures are retried, determines whether decomposition helps or merely adds latency. A memory layer lets the system carry context across turns and amortise repeated work rather than treating each query in isolation. Finally, football's defining question, who will win, demands a genuine predictive model, so we integrate a calibrated match-outcome predictor as a first-class tool, benchmarked against naive baselines.

2 Related Work

LLM-based multi-agent systems are well documented in theory. Li et al. [7] survey the field around five components: profile, perception, self-action, mutual interaction, and evolution. Yang et al. [14] locate the advantages in task decomposition, specialisation, and proprietary-data preservation. Ye et al. [15] (MAS-GPT) train an LLM to generate an executable MAS in a single inference. Our Manager, Data Analyst, and Visualizer pipeline, bounded retry loop, and rule-based Eval agent instantiate these patterns. The common limitation is evaluation: this evidence base consists of general-purpose QA and reasoning benchmarks that do not isolate orchestration from tool and database access, nor report its token and latency cost. Our design, comparing a MAS, a single LLM with tools, and a single LLM on raw data, decomposes exactly that effect, finding that structured-database access (+48 points) dwarfs the MAS orchestration gain (+6 points).

The closest domain effort is SoccerAgent [8], which contributes SoccerWiki (a multimodal knowledge base), SoccerBench (a $\sim$10K multimodal QA benchmark across 13 tasks), and a MAS that decomposes soccer questions and outperforms MLLM baselines. Its focus is multimodal understanding, whereas we target structured analytical querying together with a calibrated predictive tool. It also benchmarks only against MLLMs, omitting the controlled MAS-versus-

single-LLM-with-equal-tools ablation, the autonomy sweep (controlled, semi-controlled, autonomous), and the memory ablation that form our empirical contribution.

On prediction, prior work consists of bespoke single-model research rather than an agent tool. Constantinou [2] (Dolores) combines dynamic ratings and Hybrid Bayesian Networks across 52 leagues. Wong et al. [12] are the closest analogue to our predictor, combining standard classifiers, LightGBM, and a CNN through stacking and voting ensembles on the English Premier League. Bunker and Thabtah [1] stress chronological train/test splitting and warn against accuracy-only evaluation, and Wheatcroft and Sienkiewicz [11] argue for calibration-sensitive scoring; together these motivate our pre-kickoff features, out-of-sample testing, and use of Brier score, log loss, and isotonic calibration. Iskandaryan et al. [6] and Rossi et al. [10] add weather and GPS-injury modelling respectively.

3 Methods and Technical Decisions

3.1 Data Architecture

Our football analytics knowledge base integrates data from five complementary providers, each contributing a distinct information layer. Together they span the top five European leagues, namely the English Premier League (ENG-PL), Spanish La Liga (ESP-LL), Italian Serie A (ITA-SA), German Bundesliga (GER-BL), and French Ligue 1 (FRA-L1), across five seasons from 2020–21 through 2024–25, covering approximately 9,500 fixtures and 6,500 unique players.

WhoScored provides event-level data (passes, shots, tackles, dribbles, fouls, duels, set pieces, goalkeeper actions) with match, player, and team identifiers, timing, pitch coordinates, and metadata. A key contribution is decomposing WhoScored's proprietary qualifier structure into directly queryable columns, removing the need for downstream agents to parse raw qualifier strings; its `game_id` is the main match-level join key. Understat contributes player-level attacking data including goals, assists, shots, minutes, xG, xA, npxG, xGChain, and xGBuildup, split by position, situation, and shot zone or type. FBref is scraped at match and season level, providing per-match team, player, and goalkeeper statistics and lineups, plus season-level squad and opponent aggregates and league standings (home/away splits, attendance, top scorers), connecting individual performance to season context. Transfermarkt adds availability and market-value data: player profiles, market-value snapshots, transfer histories, and injury records (reason, dates, days and games missed). Football-Data.co.uk supplies validation statistics (shots, corners, cards).

The knowledge base is a three-layer relational schema, separating ingestion, entity standardisation, and query-serving. This lets downstream agents query stable, documented, semantically consistent tables rather than reasoning over heterogeneous raw files or provider-specific naming, which improves reliability and reduces query complexity.

The same player, club, or match appears under different names and identifiers across providers, so we resolve entities through deterministic mappings, fuzzy matching, and manual validation.

We maintain a YAML specification of the knowledge base optimised for injection into agent context, so that agents do not need to reverse-engineer the database on each query. It is organised into tables, views, and relationships: each table and view records its columns (types and plain-English descriptions), key constraints, partitioning, and a purpose field describing the questions it answers; the relationships section encodes foreign-key chains as named join paths, letting the Data Discovery agent choose joins without inspecting the raw schema. The specification is loaded once at startup and injected into the Data Discovery node's prompt, grounding table selection in the actual database structure rather than the agent's prior assumptions.

3.2 MAS Architecture

The Manager Agent decomposes the query intent, plans which agents to involve in the execution, and extracts structured parameters (teams, seasons, metrics). Deterministic queries route to the data-analysis branch, forecasts to the prediction agent, and out-of-scope inputs directly to the synthesiser. This gives a well-defined per-query path. A Data Discovery agent identifies the relevant schema tables and columns; the Data Analyst agent then generates and executes SQL against the database. Each generated query is validated against its result; on failure, the Data Analyst retries once with a corrective hint, in a bounded loop of at most two attempts. The Synthesiser composes a natural-language answer; a rule-based Eval agent then checks it against the retrieved figures, adding no model calls. Inconsistencies are flagged and, on a detected error, written to the corrections store for future reuse. Before the pipeline runs, a Memory Retrieval agent queries the stores for semantically similar prior turns and reusable cached SQL; afterwards, the completed turn is persisted. A thumbs up/down control records user feedback against each turn, informing future routing and evaluation.

Services. Agent orchestration is handled by LangGraph, which defines the agent graph, manages routing between nodes, and propagates a shared state across the pipeline. To route requests across multiple frontier models through a single interface, we use LiteLLM, which normalises each provider's request and response format (including function-calling) into one provider-agnostic interface and lets us swap models via configuration rather than code changes. Models are served by the commercial APIs of OpenAI, Anthropic, and Google: GPT-5.5, Claude Opus 4.8, and Gemini 3.1 Pro Preview. For observability we instrument the system with Langfuse, attached as a LiteLLM callback and exporting traces over OpenTelemetry; this provides the per-run token-usage and latency measurements reported here. Our analytical data resides in a PostgreSQL database hosted on Aiven, which the Data Analyst agent queries through SQLAlchemy. The memory layer reuses the same Aiven Postgres instance via

the `pgvector` extension, storing conversation history, a cache of successful SQL queries, and a record of past corrections; embeddings are generated locally with the `all-MiniLM-L6-v2` sentence-transformer model. The conversational interface is served with Chainlit, and the application is deployed on Modal for serverless hosting.

Tools. The agents access three purpose-built tools, each exposed as a callable function rather than as direct access to any backend. The `use_template` tool offers a catalogue of predefined, parameterised SQL queries covering the most common analytical questions, and is the primary tool available to the controlled and semi-controlled variants. The `run_sql` tool lets the autonomous variant (and the semi-controlled variant as a fallback) issue free-form SQL against the football database, guarded by a read-only validation layer that rejects any non-SELECT statement. The `predict_match_outcome` tool wraps our trained match-outcome model: given a fixture (home team, away team, and date) it returns calibrated home-win, draw, and away-win probabilities, computed only from data available before kickoff to preserve a causal guarantee.

Prediction Model. We frame match-outcome prediction as a three-class problem (home win, draw, away win). The model is a calibrated stacking ensemble of logistic-regression, LightGBM, and XGBoost base learners combined by a logistic meta-learner, with isotonic calibration applied to the ensemble; well-calibrated probabilities matter for a noisy forecasting task where users must trust the outputs. We evaluate with Brier score and log loss alongside accuracy.

To avoid leakage, all pre-match features use only information available before kick-off: team-strength (Elo), rolling five-match form (points, goals for and against, shots, expected goals for and against), rest days, and team-level expected goals aggregated from player data.

The feature vector at inference must match the one seen in training. We achieve this by running the same feature-engineering pipeline in both settings: at inference, a synthetic fixture is appended to the historical match record and passed through the identical pipeline, so rolling and xG features reflect a team's true pre-match state rather than an approximation.

The model is trained on 2020–2024 and evaluated out-of-sample on the held-out 2024–2025 season, compared against a base-rate predictor that always predicts the training-set class frequencies.

3.3 Experiment Designs

Single-LLM Conditions. We evaluated two single-LLM conditions, both stripped of any multi-agent orchestration. Both share the same brief role description ("a Premier League data analyst answering only from the database") and the identical database schema given to every condition; they differ only in how the model reaches the data. In the single-LLM-with-tools condition, the model is given native function-calling access to the same two tools the autonomous MAS uses,

`run_sql` for free-form queries and `predict_match_outcome` for fixture forecasts, and answers by issuing tool calls in a loop. In the single-LLM-with-raw-data condition, the model receives no tools and no database access; instead we inline a text dump of the core modelling tables (season standings, home/away splits, team and player season statistics) directly into the system prompt and instruct it to answer only from that data, stating explicitly when the required data is absent. By design the dump excludes the largest and most specialised tables (match events, player-match performance, transfers, market values, and injuries), so questions requiring those sources are unanswerable in this condition. This constraint accounts for its low accuracy on the corresponding query categories.

Ground-Truth Dataset and Evaluation. Our ground-truth dataset comprises 50 questions spanning nine analytical categories and four difficulty tiers, from simple lookups to multi-step edge cases. Of the 50 questions, 46 are data-analysis queries scored programmatically against deterministic SQL ground truth. The remaining four are prediction-related and evaluated on probabilistic forecast quality (Brier score) and correct delegation to the prediction model, rather than on an exact-match answer.

Each of the 46 data-analysis questions is paired with a hand-verified canonical SQL query whose execution against the database yields the reference answer; the questions, queries, and results are then content-hashed and frozen, together with the originating git commit, so that the agents never see the ground truth and the benchmark cannot drift between runs.

Experiment 1: Degrees of Agent Autonomy. We compared versions of the MAS that differ only in the autonomy of the Data Analyst agent; every other component (Manager, Data Discovery, Synthesiser) and the schema context were held identical. The fully controlled version can answer only by selecting from a set of pre-specified SQL templates covering common question types. The semi-controlled version has access to those same templates but may also write its own SQL, deciding per query which approach to take and falling back to free SQL if a chosen template fails. The fully autonomous version writes all queries itself, generating two independent SQL statements and reconciling their results before answering.

Experiment 2: MAS vs. Single-LLM Variants. We tested whether multi-agent orchestration itself adds value beyond simple tool access. We fixed the MAS at its best-performing autonomy level from Experiment 1 (fully autonomous) and compared it against the two single-LLM baselines with no orchestration. The single-LLM-with-tools baseline isolates the effect of orchestration alone, since it has the same tools and schema context as the MAS. The single-LLM-with-raw-data baseline must answer purely by reasoning over inlined data. Comparing these three conditions separates the gain from tool access (raw to tools) from the additional gain from multi-agent orchestration (tools to MAS).

Table 1. Accuracy by model, variant, and coverage (data-analysis queries, $n = 46$).

Model	Variant	In-coverage (28)	Out-of-coverage (18)	Overall (46)
Opus 4.8	Controlled	26/28 (93%)	1/18 (6%)	27/46 (59%)
Opus 4.8	Semi	26/28 (93%)	16/18 (89%)	42/46 (91%)
Opus 4.8	Autonomous	23/28 (82%)	17/18 (94%)	40/46 (87%)
GPT 5.5	Controlled	26/28 (93%)	3/18 (17%)	29/46 (63%)
GPT 5.5	Semi	25/28 (89%)	16/18 (89%)	41/46 (89%)
GPT 5.5	Autonomous	23/28 (82%)	16/18 (89%)	39/46 (85%)
Gemini 3.1 Pro	Controlled	26/28 (93%)	2/18 (11%)	28/46 (61%)
Gemini 3.1 Pro	Semi	26/28 (93%)	16/18 (89%)	42/46 (91%)
Gemini 3.1 Pro	Autonomous	19/28 (68%)	13/18 (72%)	32/46 (70%)

Experiment 3: Memory-Augmented MAS. A persistent memory layer augments the MAS; we evaluate its contribution to computational efficiency. To isolate the contribution of memory, we ran a fully autonomous MAS over the ground-truth dataset and compared a memory-augmented version against a memory-disabled version, measuring accuracy, latency, and token usage. All other components are fixed between the two runs, so that the difference is attributable to memory alone. To prevent leakage, the ground-truth queries are gated so that they are never written to the correction memory during a scored run, which stops the evaluation set from influencing the system's behaviour on subsequent queries.

4 Results

4.1 Optimal Degree of MAS Autonomy

We first compared MAS versions with different degrees of autonomy. Since the SQL templates available to the controlled and semi-controlled agents are not helpful for the prediction-related queries, we limited this evaluation to the 46 data-analysis queries. We further split the dataset into in-coverage (28) and out-of-coverage (18) queries, indicating whether a query can be answered with an available SQL template or requires different query and join logic.

The experiment shows that limiting the agent to predefined templates leads to low accuracy on out-of-coverage queries, making it less useful for a production system exposed to unseen and ambiguous queries. While the highest overall accuracy was achieved by Opus 4.8 and Gemini 3.1 Pro Preview at 91%, we proceeded with the combination achieving the highest out-of-coverage accuracy, since we want to optimise for unseen and ambiguous queries. Subsequent experiments therefore use a fully autonomous MAS with Opus 4.8.

4.2 MAS vs. Single-LLM Agents

Next we tested the benefit of using a MAS versus a single LLM. To isolate the orchestration benefit, we defined two single-LLM versions, one with equal tool

Table 2. MAS vs. single-LLM on data-analysis queries ($n = 46$).

Condition	Overall	Avg tokens	Avg latency
MAS (autonomous)	41/46 (89%)	30,635	17.8 s
single-LLM + tools	38/46 (83%)	24,740	10.5 s
single-LLM + raw data	16/46 (35%)	229,749	4.5 s

access to the database and prediction function and one with access only to raw data. We then compared the fully autonomous MAS against both.

The MAS achieves the highest accuracy at 89%, consuming an average of 30,635 tokens and 17.8 seconds per answer. The single-LLM-with-tools condition is slightly lower at 83%, while being more token-efficient and faster. The single-LLM-with-raw-data condition reaches only 35% accuracy while consuming almost ten times as many tokens as the single LLM that can query the database directly.

This last figure must be read against a structural limit: the raw-data condition can only inline the smaller tables, so just 30 of the 46 questions are answerable from the provided data. The remaining 16 require tables too large to fit in the prompt (match events, transfers, injuries, and player-match data).

The results indicate that the largest gain in accuracy (48 points, from 35% to 83%) comes from tool access, principally the ability to query the SQL database directly. The additional orchestration effect of the MAS contributes a smaller improvement of 6 points (83% to 89%).

In addition to the data-analysis tasks we tested more complex queries where the agents must call our proprietary prediction model. Both the MAS and the single LLM with tool access correctly delegated to the prediction agent, generating the identical expected model output. The MAS did so while consuming fewer tokens and with lower latency. The single LLM with no model access performed its own reasoning over match-outcome probabilities and produced a worse prediction, as shown by the higher average Brier score.

4.3 Memory Augmentation on Golden-Set Queries

We test whether cache reuse is selective via a three-tier sweep. Each golden-set question is run cold, establishing a cache-free baseline, then re-run as an exact repeat, a light reword, and a near-miss (for example a swapped player or season).

On true repeats and rewords the cache hits reliably (45/46 and 44/46), cutting tokens by 44% and latency by 17% by skipping the SQL-generation call, with no accuracy loss. Near-misses expose a failure mode where cosine similarity is poorly suited to temporal data: semantic similarity is good at finding candidate matches but poor at deciding whether they are safe to reuse. Raising the threshold from 0.88 to 0.99 could yield better results but would suppress legitimate rewords, so further tuning is needed to find the right operating point.

The fix is two-stage: retrieve candidates by embedding similarity as before, but authorise reuse only when the structured parameters the Manager already

Table 3. Out-of-sample match-outcome prediction, 2024–2025 season ($n = 379$).

Model	Accuracy	Brier score
Base rate	40.9%	0.218
MA-FDSS	53.3%	0.198

extracts (team, season, metric) match exactly. This preserves paraphrase hits while categorically blocking temporal and entity swaps, and adds no new extraction cost, since those parameters are parsed upstream regardless.

4.4 Prediction Model

We evaluate the match-outcome model out-of-sample on the 379 fixtures of the held-out 2024–2025 Premier League season. The model is a calibrated stacking ensemble (logistic regression, XGBoost, and LightGBM base learners with a logistic meta-learner and isotonic calibration), trained on team-strength (Elo), recent form, and rest features. We compare it against a base-rate baseline that always predicts the most frequent class.

The model improves 12.4 points over the base rate. Group-wise permutation confirms Elo as the dominant feature (-9.4 points when permuted, versus -2.5 and -1.2 for defensive and attacking form respectively). One inherent limitation remains: under argmax the model never predicts a draw (24.5% of fixtures), as draws lack a distinctive feature signature, and draw-favouring decision rules recover recall but degrade accuracy at every threshold.

5 Limitations

Our work so far focuses on textual data in the knowledge base and a small set of tools the MAS can access. A natural extension is to move towards a multimodal system and to connect more tools to enhance the MAS's capabilities.

Several caveats apply to our evaluation. The golden set is small, and accuracy is reported on a single run rather than averaged across repeats, so differences of one to three questions are within noise.

The memory layer's benefit is unproven on accuracy: our golden set comprises independent queries, so retrieval has no prior relation to surface, and the correction mechanism, though functional, has no naturally recurring failure pattern to act on, since every failure is a singleton. Its demonstrated value is cost amortisation on repeated queries; session-structured evaluation is needed to test conversational benefit.

Similarity-based cache reuse is unsafe for temporal data at the production threshold: near-identical queries differing only by season embed at $\sim$0.99 and produce false hits, serving stale SQL. Reuse should be gated on an exact match of extracted parameters (team, season, metric), which we leave to future work. We also note that the correction store's write path conflates transient API failures with genuine errors, requiring a more discriminating error detector.

References

1. Bunker, R.P., Thabtah, F.: A machine learning framework for sport result prediction. Applied Computing and Informatics **15**(1), 27–33 (2019)
2. Constantinou, A.C.: Dolores: A model that predicts football match outcomes from all over the world. Machine Learning **108**(1), 49–75 (2019)
3. Decroos, T., Bransen, L., Van Haaren, J., Davis, J.: Actions speak louder than goals: Valuing player actions in soccer. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. pp. 1851–1861 (2019)
4. Gulli, A.: Agentic Design Patterns: A Hands-On Guide to Building Intelligent Systems (2025)
5. Hong, S., Zheng, X., Chen, J., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S.K.S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C.: Metagpt: Meta programming for a multi-agent collaborative framework. In: International Conference on Learning Representations (ICLR) (2024)
6. Iskandaryan, D., et al.: The effect of weather in soccer results: An approach using machine learning techniques. Applied Sciences **10**(19) (2020)
7. Li, X., et al.: A survey on llm-based multi-agent systems: Workflow, infrastructure, and challenges. arXiv preprint arXiv:2402.01680 (2024)
8. Rao, J., et al.: Towards universal soccer video understanding with a multi-agent system. In: Proceedings of the 33rd ACM International Conference on Multimedia (MM '25) (2025)
9. Rein, R., Memmert, D.: Big data and tactical analysis in elite soccer: Future challenges and opportunities for sports science. SpringerPlus **5**(1), 1410 (2016)
10. Rossi, A., Pappalardo, L., Cintia, P., Iaia, F.M., Fernández, J., Medina, D.: Effective injury forecasting in soccer with gps training data and machine learning. PLOS ONE **13**(7), e0201264 (2018)
11. Wheatcroft, E., Sienkiewicz, E.: Forecasting and evaluating football match outcomes using probabilistic scoring rules. International Journal of Forecasting (2021)
12. Wong, A., et al.: Forecasting english premier league match outcomes using stacking and voting ensembles. Expert Systems with Applications (2024)
13. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A.H., White, R.W., Burger, D., Wang, C.: Autogen: Enabling next-gen llm applications via multi-agent conversation. In: arXiv preprint arXiv:2308.08155 (2023)
14. Yang, H., et al.: Multi-agent systems powered by large language models: Applications, advantages, and challenges. arXiv preprint (2024)
15. Ye, R., et al.: Mas-gpt: Training llms to build multi-agent systems in a single inference. arXiv preprint (2025)
16. Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., Radev, D.: Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 3911–3921 (2018)

A Experimental Setup and Results

This appendix provides the full experimental detail behind the results in the main text: system diagrams, the complete golden-set questions, model configu-

ration, aggregate and per-question result tables, the failure taxonomy and root-cause analysis, and the exact prompts used for each condition.

A.1 System Diagrams

Figure 1 shows the full agent pipeline, and Figure 2 shows the memory layer and its read/write paths into the three pgvector stores.

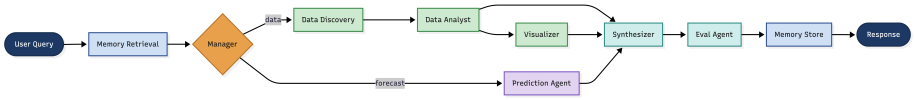


Fig. 1. MA-FDSS agent pipeline. The Manager routes each query to the data-analysis branch (Data Discovery → Data Analyst → Visualizer) or to the Prediction Agent; both converge on the Synthesiser, followed by the Eval Agent and Memory Store.

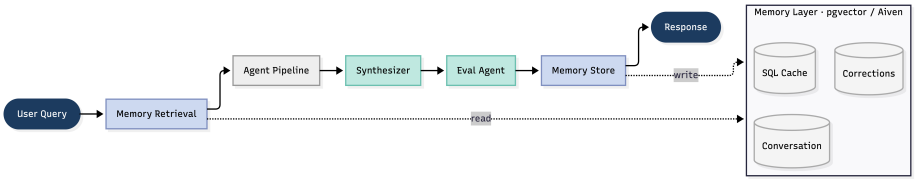


Fig. 2. Memory layer. The main pipeline (solid) reads from the memory stores before execution and writes the completed turn afterward (dashed). The layer comprises conversation memory, an SQL cache, and a corrections store, all hosted on the same pgvector/Aiven instance.

A.2 Data Ingestion Sources

Figure 3 summarises the five data providers, their granularity, key fields, and approximate row counts after ingestion into the raw layer.

Layer	Source	Granularity	Key fields	~Rows
Match events	WhoScored	Per event	Event type, pitch coords (x/y/end_x/end_y), timing (min/sec/expanded), player & team info, decomposed qualifiers: pass type ×5, shot qualifier ×5, keeper save type ×4, foul type, body part, blocked cross	~7.5M
Fixture schedule	WhoScored	Per match	game_id, home/away team, kickoff time, venue — primary join key across pipeline	~9,500
Player career stats	Understat	Per player-season	Goals, assists, shots, minutes, non-penalty goals (npg), xG, xA, npxG, yellow/red cards	~70K
Player match stats	Understat	Per player-match	xG, xA, npxG, xGChain, xGBuildup, key passes, goals, assists	~245K
Individual shots	Understat	Per shot	xG, situation (open play / set piece / counter / penalty), shot type, last action, player assisted	~430K
Player season splits	Understat	Per player-season-context	Stats split by position, match situation, shot zone, and shot type	~170K
Match metadata	FBref (HTML)	Per match	Lineups, formations, substitutions, referee, attendance, half-time scores	~9,500
Player match stats	FBref (HTML)	Per player-match	Goals, assists, shots, progressive passes, tackles won, interceptions, fouls committed/drawn, xG, xA, saves, goals conceded	~190K
Team season standings	FBref	Per team-season	Points, goal difference, home/away splits, top scorer, average attendance	~500
Team squad stats	FBref	Per team-season	Squad & opponent aggregates: standard, shooting, passing, defensive, GK tables	~1,000
Player profiles	Transfermarkt	Per player	Date of birth, nationality, height, preferred foot, current club, position	~6,500
Market values	Transfermarkt	Per player-date	Weekly valuation snapshots across full study window	~130K
Transfers	Transfermarkt	Per transfer	Origin/destination club, transfer fee, type, market value at transfer date	~50K
Injury spells	Transfermarkt	Per spell	Injury reason, from/end date, matchweeks missed, calendar days missed, games missed	~25K
Match stats & odds	Football-data	Per match	Shots, shots on target, corners, fouls, yellow/red cards (cross-validation); 1X2, Over/Under 2.5, Asian handicap odds from 12 bookmakers (opening + closing); average market prices used as pre-match probability features	~9,500

Fig. 3. Data ingestion sources. Five providers span event-, player-, team-, and match-level granularity across the top-five European leagues and five seasons; row counts are approximate post-ingestion totals.

A.3 Golden-Set Questions (data analysis, $n = 46$)

The complete canonical SQL for each question is in `results/golden/ground_truth.json`.

Table 4: Golden-set data-analysis questions ($n = 46$), with category, difficulty tier, and template coverage.

ID	Category	Difficulty	Cov	Question
Q01	team_league_outcomes	simple	in	Which team won the Premier League in 2024-2025?
Q02	team_league_outcomes	simple	in	How many points did Arsenal finish with in 2023-2024?
Q03	team_league_outcomes	medium	in	Full 2022-2023 PL final table (rank, team, points).
Q04	team_league_outcomes	medium	in	Which three teams were relegated in 2023-2024?
Q05	team_league_outcomes	medium	in	Best goal difference in 2024-2025?
Q06	team_league_outcomes	edge	out	Everton 2023-2024 points with vs without deduction?
Q07	team_league_outcomes	hard	out	Highest single-season points total, 2020-2025?
Q08	team_league_outcomes	simple	in	Teams that finished top four in 2021-2022?
Q09	player_season_aggregates	simple	in	Top goalscorer in 2023-2024 and goals?
Q10	player_season_aggregates	medium	in	Top five goalscorers in 2024-2025 with totals.

continued on next page

ID	Category	Difficulty	Cov	Question
Q11	player_season_aggregates	medium	in	Most assists in 2023-2024?
Q12	player_season_aggregates	edge	in	Cole Palmer total 2023-2024 goals (all clubs)?
Q13	player_season_aggregates	medium	in	Player with most minutes in 2022-2023?
Q14	player_season_aggregates	medium	in	Liverpool top scorer in 2024-2025?
Q15	player_season_aggregates	medium	in	Players with ≥20 league goals in 2024-2025?
Q16	home_away_splits	medium	in	Most home points in 2024-2025?
Q17	home_away_splits	medium	in	Most away wins in 2023-2024?
Q18	home_away_splits	hard	out	Biggest home-vs-away points gap in 2023-2024?
Q19	home_away_splits	simple	in	Arsenal home points in 2023-2024?
Q20	home_away_splits	medium	in	Most home goals in 2024-2025?
Q21	disciplinary	medium	in	Most yellow cards (team) in 2023-2024?
Q22	disciplinary	medium	in	Most yellow cards (player) in 2023-2024?
Q23	disciplinary	medium	in	Most red cards (team) in 2024-2025?
Q24	disciplinary	medium	out	Total red cards league-wide in 2023-2024?
Q25	disciplinary	edge	out	PL matches refereed by Anthony Taylor in 2023-2024?
Q26	xg_performance	hard	out	Biggest xG overperformer (≥10 goals) 2023-2024?
Q27	xg_performance	hard	out	Biggest xG underperformer (≥10 goals) 2023-2024?
Q28	xg_performance	medium	out	Haaland 2023-2024 goals vs xG and difference?
Q29	xg_performance	hard	out	Highest goals-to-xG ratio (≥15 xG) 2024-2025?
Q30	xg_performance	hard	out	Team with most combined npxG in 2023-2024?
Q31	market_transfers	medium	in	Player with highest most-recent market value?
Q32	market_transfers	hard	in	Five largest incoming transfer fees to PL clubs?
Q33	market_transfers	medium	in	Most expensive incoming transfer, 2023-2024 window?
Q34	market_transfers	hard	out	Chelsea total incoming transfer spend (all seasons)?
Q35	market_transfers	medium	in	Three highest market-value players at Man City?
Q36	availability	medium	out	Most games missed in a single injury spell, 2023-2024?
Q37	availability	hard	out	Most total days missed to injury, 2023-2024?
Q38	availability	hard	out	Club with most separate injury spells, 2023-2024?
Q39	availability	medium	out	Injury spells recorded for Brentford players, 2023-2024?

continued on next page

ID	Category	Difficulty	Cov	Question
Q40	event_player_match	hard	in	Most shots attempted (event data) in 2023-2024?
Q41	event_player_match	medium	in	Most goals by one player in a single match, 2024-2025?
Q42	event_player_match	hard	out	Most fouls committed (summed) in 2023-2024?
Q43	event_player_match	medium	out	Shot events resulting in a goal in 2024-2025?
Q44	event_player_match	medium	in	Biggest home win (margin) in 2024-2025?
Q45	edge_case	edge	in	Norwich City points in 2024-2025? (expect denial)
Q46	edge_case	edge	out	Matches in 2020-2021 with a recorded attendance figure?

Prediction questions ($n = 4$). These are scored on (a) the Brier score of the predicted home/draw/away probabilities against the realised outcome, and (b) fidelity, namely whether the agent correctly delegated to the prediction model and reproduced its output.

Table 5. Prediction (forecast) questions ($n = 4$) with model ground-truth probabilities.

ID	Diff.	Question	Ground truth (H/D/A)
P01	medium	Man United vs Fulham, 2024-08-16	0.557 / 0.178 / 0.265
P02	medium	Arsenal vs Wolves, 2024-08-17, with confidence	0.800 / 0.184 / 0.016
P03	medium	Everton vs Brighton, 2024-08-17	0.239 / 0.224 / 0.538
P04	hard	Draw prob., Nottingham Forest vs Bournemouth, 2024-08-17	0.322 / 0.249 / 0.429

A.4 Model IDs and Configuration

Table 6. Model identifiers and temperature configuration per logical family.

Logical family	LiteLLM model ID	Temperature
claude	`anthropic/claude-opus-4-8`	model default (omitted)
gpt	`gpt-5.5`	model default (omitted)
gemini	`gemini/gemini-3.1-pro-preview`	model default (omitted)

Routing. All model calls go through LiteLLM (`completion`), giving one OpenAI-style interface (including cross-provider function-calling) across the three families. **Temperature.** All three frontier models deprecate or restrict the `temperature` parameter, so we omit it and let each model use its own default. SQL-generating paths request 0.3, but this is suppressed for these models; this is a methodological constraint rather than a fixed temperature. **Max tokens.** 4096, generous enough that thinking models return content rather than only reasoning. **Memory.** Disabled for all evaluation runs (no read or write) so the comparison isolates orchestration. **Tokens and latency.** Captured locally via LiteLLM usage accounting per run, except MAS data-analysis token totals, which were recovered from Langfuse traces because local capture does not see the data-analyst self-debate loop (see the Caveats subsection).

A.5 Aggregate Result Tables

Table 7. Experiment 1 (degrees of autonomy): accuracy by model, variant, and coverage ($n = 46$).

Model	Variant	In-cov (28)	Out-of-cov (18)	Overall (46)
Opus 4.8	Controlled	26/28 (93%)	1/18 (6%)	27/46 (59%)
Opus 4.8	Semi	26/28 (93%)	16/18 (89%)	42/46 (91%)
Opus 4.8	Autonomous	23/28 (82%)	17/18 (94%)	40/46 (87%)
GPT 5.5	Controlled	26/28 (93%)	3/18 (17%)	29/46 (63%)
GPT 5.5	Semi	25/28 (89%)	16/18 (89%)	41/46 (89%)
GPT 5.5	Autonomous	23/28 (82%)	16/18 (89%)	39/46 (85%)
Gemini 3.1 Pro	Controlled	26/28 (93%)	2/18 (11%)	28/46 (61%)
Gemini 3.1 Pro	Semi	26/28 (93%)	16/18 (89%)	42/46 (91%)
Gemini 3.1 Pro	Autonomous	19/28 (68%)	13/18 (72%)	32/46 (70%)

Experiment 1: Degrees of freedom.

Table 8. Experiment 2: MAS vs. single-LLM on data-analysis queries ($n = 46$).

Condition	Overall	In-cov (28)	Out-of-cov (18)	Avg tokens	Avg latency
MAS (autonomous)	41/46 (89%)	24/28 (86%)	17/18 (94%)	30,635	17.8 s
single-LLM + tools	38/46 (83%)	23/28 (82%)	15/18 (83%)	24,740	10.5 s
single-LLM + raw	16/46 (35%)	13/28 (46%)	3/18 (17%)	229,749	4.5 s

Experiment 2: MAS vs single-LLM (Claude), data analysis. Accuracy decomposition: tool access contributes +48 points (35% to 83%); additional multi-agent orchestration +6 points (83% to 89%).

Table 9. Experiment 2: accuracy by query category.

Category	MAS	single + tools	single + raw
availability	4/4	4/4	0/4
disciplinary	4/5	5/5	3/5
edge_case	2/2	1/2	1/2
event_player_match	3/5	4/5	0/5
home_away_splits	5/5	4/5	4/5
market_transfers	4/5	1/5	0/5
player_season_aggregates	6/7	7/7	1/7
team_league_outcomes	8/8	8/8	7/8
xg_performance	5/5	4/5	0/5

Table 10. Experiment 2: prediction slice ($n = 4$).

Condition	Avg Brier	Fidelity	Avg tokens	Avg latency
MAS (autonomous)	0.129	4/4	992	8.5 s
single-LLM + tools	0.129	4/4	34,947	12.2 s
single-LLM + raw data	0.166	n/a	1,156	6.4 s

Experiment 2: prediction slice (Claude, $n = 4$). MAS and single-LLM-with-tools produce identical predictions (both delegate to the same model function), hence identical Brier scores. The raw-data condition reasons over recent results with no model access and is less calibrated.

A.6 Per-Question Matrix: Experiment 1 (DoF)

`1` = correct, `0` = incorrect. Columns: model (clau/gpt/gemi) × variant (con/sem/aut).

Table 11. Per-question correctness, Experiment 1. Columns: model (clau/gpt/gemi) × variant (con/sem/aut).

QID	Cov	cl.con	cl.sem	cl.aut	gp.con	gp.sem	gp.aut	ge.con	ge.sem	ge.aut
Q01	in	1	1	1	1	1	1	1	1	1
Q02	in	1	1	1	1	1	1	1	1	1
Q03	in	1	1	0	1	1	1	1	1	0
Q04	in	1	1	1	1	1	1	1	1	0
Q05	in	1	1	1	1	1	1	1	1	1
Q06	out	0	1	1	1	1	1	1	1	1
Q07	out	0	1	1	1	1	1	0	1	0
Q08	in	1	1	1	1	1	0	1	1	0
Q09	in	1	1	1	1	1	1	1	1	1
Q10	in	1	1	1	1	1	1	1	1	1
Q11	in	1	1	1	1	1	1	1	1	1
Q12	in	1	1	1	1	1	1	1	1	1
Q13	in	1	1	0	1	1	0	1	1	0
Q14	in	1	1	1	1	1	1	1	1	1
Q15	in	1	1	1	1	1	1	1	1	1
Q16	in	1	1	1	1	1	1	1	1	0
Q17	in	1	1	1	1	1	1	1	1	1
Q18	out	0	1	1	0	1	1	0	1	1
Q19	in	1	1	1	1	1	1	1	1	1
Q20	in	1	1	1	1	1	1	1	1	0
Q21	in	1	1	1	1	1	1	1	1	1
Q22	in	1	1	1	1	1	1	1	1	1
Q23	in	1	1	1	1	1	1	1	1	1
Q24	out	1	1	1	1	1	1	1	1	1
Q25	out	0	0	0	0	0	0	0	0	0
Q26	out	0	1	1	0	1	1	0	0	1
Q27	out	0	1	1	0	1	1	0	1	0
Q28	out	0	1	1	0	1	1	0	1	1
Q29	out	0	1	1	0	1	1	0	1	1
Q30	out	0	1	1	0	1	1	0	1	1
Q31	in	1	1	1	1	1	1	1	1	1
Q32	in	0	0	1	0	0	0	0	0	1
Q33	in	1	1	1	1	1	1	1	1	1
Q34	out	0	1	1	0	0	0	0	1	1
Q35	in	1	1	0	1	1	0	1	1	0
Q36	out	0	1	1	0	1	1	0	1	0
Q37	out	0	1	1	0	1	1	0	1	1
Q38	out	0	1	1	0	1	1	0	1	0
Q39	out	0	1	1	0	1	1	0	1	1
Q40	in	1	1	0	1	1	1	1	1	0
Q41	in	0	0	0	0	0	0	0	0	0
Q42	out	0	0	1	0	1	1	0	1	1
Q43	out	0	1	1	0	1	1	0	1	1
Q44	in	1	1	1	1	1	1	1	1	1
Q45	in	1	1	1	1	0	1	1	1	1
Q46	out	0	1	1	0	1	1	0	1	1

A.7 Per-Question Matrix: Experiment 2 (MAS vs single, Claude)

Table 12. Per-question correctness, Experiment 2 (Claude): MAS vs. single-LLM with tools vs. single-LLM with raw data.

QID	Cov	MAS	+tools	+raw	QID	Cov	MAS	+tools	+raw
Q01	in	1	1	1	Q24	out	1	1	1
Q02	in	1	1	1	Q25	out	0	1	0
Q03	in	1	1	1	Q26	out	1	1	0
Q04	in	1	1	1	Q27	out	1	1	0
Q05	in	1	1	1	Q28	out	1	0	0
Q06	out	1	1	1	Q29	out	1	1	0
Q07	out	1	1	0	Q30	out	1	1	0
Q08	in	1	1	1	Q31	in	1	0	0
Q09	in	1	1	0	Q32	in	1	0	0
Q10	in	1	1	0	Q33	in	1	0	0
Q11	in	1	1	0	Q34	out	1	0	0
Q12	in	1	1	0	Q35	in	0	1	0
Q13	in	0	1	0	Q36	out	1	1	0
Q14	in	1	1	1	Q37	out	1	1	0
Q15	in	1	1	0	Q38	out	1	1	0
Q16	in	1	1	1	Q39	out	1	1	0
Q17	in	1	1	1	Q40	in	0	1	0
Q18	out	1	1	1	Q41	in	0	0	0
Q19	in	1	1	1	Q42	out	1	1	0
Q20	in	1	0	0	Q43	out	1	1	0
Q21	in	1	1	1	Q44	in	1	1	0
Q22	in	1	1	0	Q45	in	1	1	1
Q23	in	1	1	1	Q46	out	1	0	0

A.8 Failure Taxonomy (semi-controlled variant)

A = chose a template when SQL was needed; B = wrong template or params; C = template failed then SQL fallback, still wrong; D = free-SQL generation wrong; E = errored or unclassified.

Table 13. Failure taxonomy for the semi-controlled variant, by model and failure mode.

Code	Failure mode	claude	gpt	gemini	Total
A	chose template when SQL needed	0	0	0	0
B	wrong template / params	2	3	2	7
C	template failed → SQL fallback, wrong	0	0	0	0
D	free-SQL generation wrong	2	2	2	6
E	errored / unclassified	0	0	0	0

A.9 Root-Cause Analysis of Failures

Controlled variant collapses on out-of-coverage (6–17%). By construction the template catalogue (see the Template catalogue under Exact Prompts

per Condition) cannot express computed columns, `HAVING` filters, cross-season aggregation, or the events, transfers, and injuries tables. Every out-of-coverage miss (Q06, Q07, Q18, Q26–Q30, Q34, Q36–Q39, Q42, Q43, Q46) is a no-template-fits failure, which is the expected result of constraining the agent to templates and the central motivation for allowing free SQL.

Universal failures (all variants, all models). Two questions are missed by every configuration. Q41 (most goals by one player in a single match): `player_match_performance` has a per-(player, game) grain that all systems mis-aggregated, and the matching template returns a different shape than the ground truth expects; this is the only in-coverage question the controlled variant fails for all models and is a likely ground-truth grain mismatch worth re-auditing. Q25 (matches refereed by Anthony Taylor): the `referee` column contains a non-breaking space (`chr(160)`) that the canonical SQL normalises with `replace(...)`; agents that wrote the natural `referee = 'Anthony Taylor'` returned zero rows. This is a data-quality landmine rather than a reasoning failure, and single-LLM-with-tools alone recovered it.

Autonomous loses some in-coverage questions (the 82%/68% dip). On questions a template would have answered cleanly, free SQL occasionally diverges: Q03 and Q08 (table formatting and ordering), Q13 (minutes aggregation), Q20, Q35 (market value joined to club), and Q40. For Gemini the autonomous self-debate loop frequently failed to converge (in-coverage 19/28), the main driver of its 70% overall. This is the cost of autonomy: more expressive, but with more ways to be subtly wrong.

Semi-controlled failures split evenly. Seven are wrong-template or param (route = template: mostly Q41, Q32) and six are wrong-SQL (route = sql: Q25, Q42, Q34, Q26). Bucket A is empty, so no model ever picked a template when SQL was strictly required; the routing decision itself is sound and the errors are downstream.

Single-LLM-with-raw-data (35%) is bounded twice over. First, a coverage ceiling of 30/46: only four tables are inlined (standings, home/away splits, team season stats, player season stats), and the other 16 questions need tables too large to inline, so they are unanswerable by construction (the entire availability, market/transfer, and event categories score 0). Second, a reasoning shortfall within reach: even among the 30 answerable questions it scored only 16 (53%), with the gap concentrated in `player_season_aggregates` (1/7) and `xg_performance` (0/5), both requiring aggregation over the largest inlined table.

A.10 Exact Prompts per Condition

All conditions share the same role preamble and schema context (`get_schema_context()`), injected where indicated by `{schema}`. Variable interpolations are shown in `{braces}`.

Shared preamble (all agents).

```
You are a Premier League data analyst. Answer the user's question using
```

```
ONLY the database. Rules: seasons are 'YYYY-YYYY' (hyphen, e.g.
'2023-2024'); the latest season is 2024-2025 (no 2025-26 data); point
deductions are already in the points columns; prefer IDs over names for
joins; canonical team names (e.g. 'Manchester United', 'Wolves').
```

MAS Manager (intent + entity extraction).

```
{preamble}
Classify and extract for this Premier League question.
Question: {question}
Respond JSON: {"intent": "data_request"|"prediction_match"|"conversational",
"seasons": ["YYYY-YYYY", ...], "teams": [...], "metrics": [...],
"home_team": "...", "away_team": "...", "match_date": "YYYY-MM-DD"}.
Use intent='prediction_match' ONLY when asked to forecast a specific
fixture's outcome; then fill home_team, away_team and match_date.
Resolve 'this/current season' to 2024-2025 (latest data).
```

MAS Data Discovery.

```
{preamble}
Database schema:
{schema}
Question: {question}
Manager context: {manager_json}
Respond JSON: {"relevant_tables": [...], "notes": "short join/grain guidance"}.
```

MAS Data Analyst, CONTROLLED (template only).

```
{preamble}
You can ONLY answer using these templates:
{template_catalogue}
Question: {question}
Respond with JSON: {"template_name": "...", "params": {"season": "...",
"team": "...", "metric": "...", "top_n": 50}}. Include only the params
the template needs.
```

On template error: one corrective retry with `(Previous attempt failed: {error})` appended.

MAS Data Analyst, SEMI (route template-or-SQL).

```
{preamble}
You may either USE A TEMPLATE or WRITE CUSTOM SQL.
Templates:
{template_catalogue}
Question: {question}
Respond JSON: {"approach": "template"|"sql", "template_name": "...",
"params": {...}} (if template). If "sql", just {"approach":"sql"}.
```

If approach = "sql", or a chosen template errors, control passes to the self-debate SQL loop.

MAS Data Analyst, AUTONOMOUS (free SQL + self-debate). Two independent candidates are generated per iteration (up to 2 reconcile rounds); their result sets must agree or the answer is flagged low-confidence.

```
# Candidate A
{preamble}
Database schema:
{schema}
Question: {question}
Write the most direct correct query.
Return ONLY one SQL SELECT statement, no prose, no code fences.

# Candidate B
... Write an independent query (different phrasing/joins) for the same answer.

# Reconcile round (if A and B disagree)
... Your two previous SQL attempts DISAGREED on their results -- reconcile them.
Attempt A: {sql_a}
Attempt B: {sql_b}
Find the correct query.
```

MAS Synthesiser (data questions).

```
{preamble}
Question: {question}
Query result (JSON):
{first_30_rows}
Answer the question directly and concisely using ONLY this result.
State numbers exactly.
```

MAS Prediction synthesiser.

```
{preamble}
Question: {question}
The match-outcome model returned (probabilities):
{prediction_json}
Answer the question using these probabilities. State the three
probabilities (home win / draw / away win) explicitly and the most
likely outcome.
```

Single-LLM + tools (system prompt). Native function-calling; tools = `run_sql`, `predict_match_outcome`; max 8 steps.

```
{preamble}
Use the provided tools to answer: ‘run_sql‘ to query the database, and
‘predict_match_outcome‘ to forecast a fixture’s result. Then answer.
Database schema:
{schema}
```

Single-LLM + raw data (system prompt). No tools; the four core tables are inlined as CSV.

```
{preamble}
You have NO tools and NO database access. Answer ONLY from the schema
and the inlined data below. If the data needed is not present, say so.
Schema:
{schema}
=== INLINED DATA ===
{csv_dump_of: standings, home_away, team_discipline, player_season}
```

Single-LLM + raw data (prediction variant).

```
{preamble}
You have NO tools and NO prediction model -- forecast using ONLY the
recent results below. Reason about form, then give CALIBRATED
probabilities for {home_team} (home) vs {away_team} on {match_date}
that sum to 1.0.
=== RECENT RESULTS ===
{home_team and away_team's last 10 matches strictly before match_date}
End your answer with EXACTLY this line:
PROBABILITIES: home=<p>, draw=<p>, away=<p>
```

Tool: predict_match_outcome (schema shown to the model).

```
name: predict_match_outcome
description: Predict probabilities of each outcome (home win / draw /
 away win) for a Premier League fixture, using the trained match-outcome
 model. Returns three probabilities summing to 1.0 plus the model's
 confidence. Provide the match_date (the model is causal -- it uses only
 data before that date).
input: { home_team (req), away_team (req), match_date 'YYYY-MM-DD' (req),
 season (opt) }
```

Template catalogue (controlled / semi).

```
- league_table: Full final league table for a season. (season)
- team_season_summary: One team's season summary incl. home/away points
  & goals, possession. (team, season)
- home_away_table: Teams ranked by a home/away split metric.
  (season; metric in home_points|away_points|home_wins|away_wins|
   home_goals_for|away_goals_for)
- player_season_leaders: Top players by a season stat.
  (season; metric in goals|assists|minutes|yellow_cards|red_cards|shots)
- player_team_leaders: Top players at a club by a season stat.
  (team, season; metric in goals|assists|minutes)
- team_stat_leaders: Teams ranked by a team-season stat.
  (season; metric in yellow_cards|red_cards|goals|shots|possession)
- head_to_head: All matches between two teams in a season.
```

```
  (team_a, team_b, season)
- match_results: Matches in a season ordered by home win margin. (season)
- market_value_leaders: Players ranked by latest market value. (none)
- market_value_team_leaders: Players at a club ranked by latest value. (team)
- player_match_leaders: Top single-match player performances.
  (season; metric in match_goals|match_assists)
- transfer_leaders: Largest incoming transfer fees to PL clubs. (none)
```

A.11 Caveats

Single-repeat ($n = 46 + 4$); no variance estimates, so differences of one to three questions are within noise. MAS data-analysis token totals come from Langfuse; all other token totals come from local capture, so the two are not directly comparable across tables (the MAS prediction total of 992 is local capture and undercounts relative to its Langfuse data-analysis total). Q41 and Q25 ground truth should be re-audited (grain mismatch and non-breaking-space normalisation). The on-disk `results/stats/paired_tests.json` predates the current scorer and disagrees with the re-scored tables above; regenerate before citing any significance test.